

BYGG DIN EGEN AI-ASSISTENT MED GPT BYGGGAREN

Från idé till fungerande GPT – utan att behöva
bli expert på promptdesign och GPT-arkitektur



ERLAND LINDMARK

Innehåll

Inledning	2
1. Vad är egentligen en AI-assistent?	5
2. Från problem till GPT-idé	10
3. GPT Byggaren	15
4. Bygg din första GPT	20
5. Vad GPT Byggaren bygger åt dig	26
6. Testa, förbättra och fortsatt utveckla	32
7. Från experiment till riktig assistent	37

Inledning

ChatGPT är lätt att börja använda. Du ställer en fråga, får ett svar och fortsätter samtalet därifrån. För många uppgifter räcker det långt.

Men efter ett tag uppstår ofta ett annat behov. Du kanske återkommer till samma typ av analys, använder samma instruktioner, bifogar liknande underlag och vill ha resultatet presenterat på ungefär samma sätt varje gång. Då är det naturligt att gå från en vanlig konversation till en mer specialiserad AI-assistent.

Den här boken handlar om hur du gör det med hjälp av **GPT Byggaren**.

Målet är inte att göra dig till expert på promptdesign, scheman, testautomatisering eller intern GPT-arkitektur. Målet är att du ska förstå vad en specialiserad AI-assistent är, kunna beskriva vad du vill att den ska hjälpa till med och sedan praktiskt kunna använda GPT Byggaren för att ta fram den.

Vad du kommer att lära dig

När du har läst boken ska du kunna:

- skilja mellan vanlig användning av ChatGPT och en specialiserad AI-assistent,
- formulera ett konkret användningsfall som lämpar sig för en GPT,
- använda GPT Byggaren för att analysera behovet och skapa en utvecklingsplan,
- bygga och vidareutveckla en GPT steg för steg i en vanlig ChatGPT-konversation,
- förstå de viktigaste delarna som GPT Byggaren skapar åt dig,
- testa och förbättra assistenten utifrån verklig användning,
- avgöra när en Chat ZIP räcker och när andra distributionsformer kan vara intressanta.

Det centrala genom hela boken är arbetsfördelningen mellan dig och GPT Byggaren.

Du behöver framför allt beskriva **vad du vill åstadkomma**. GPT Byggaren hjälper till att avgöra **hur GPT-projektet bör utformas, struktureras, testas och paketeras**.

Vem boken är till för

Boken är skriven för dig som redan har använt ChatGPT men som inte behöver ha byggt en egen GPT tidigare.

Du förväntas kunna starta en konversation, skriva instruktioner med vanligt språk och bifoga en fil. Du behöver däremot inte kunna YAML, JSON Schema, GitHub Actions eller testautomatisering.

Tekniska begrepp introduceras först när de behövs. När vi tittar på exempelvis instruktioner, kunskapsunderlag, förmågor och tester är syftet att du ska förstå deras roll och kunna bedöma resultatet – inte att du ska behöva bygga allt manuellt.

Bokens huvudspår: GPT Byggaren i Chat-läge

GPT Byggaren kan användas på olika sätt. I den här boken fokuserar vi främst på **Chat-läget**.

Det innebär att du använder en Chat ZIP med GPT Byggaren i en vanlig ChatGPT-konversation. Du bifogar ZIP-filen, ber ChatGPT använda den som GPT Byggaren och beskriver sedan den assistent du vill skapa.

Arbetet blir ungefär så här:

1. Du beskriver idén och problemet du vill lösa.
2. GPT Byggaren analyserar behovet.
3. Du får en utvecklingsplan.
4. GPT Byggaren skapar ett GPT-projekt.
5. Du fortsätter utvecklingen steg för steg.
6. GPT:n testas, förbättras och paketeras för användning.

En viktig poäng är att du inte behöver hålla hela projektet i huvudet eller i chatthistoriken. Projektets struktur och status lagras i projektet. Det gör att du kan spara den senaste projekt-ZIP:en och fortsätta arbetet senare, även i en ny konversation.

Ett exempel följer med genom boken

För att göra resonemangen konkreta använder vi ett enkelt återkommande exempel.

Tänk dig att du ofta får dokument som du behöver läsa igenom för att förstå vad som är relevant för din organisation. Du vill därför skapa en assistent som kan:

- ta emot ett dokument,
- identifiera de delar som kan vara viktiga,
- sammanfatta dem kort,
- strukturera resultatet på ett konsekvent sätt.

Det är medvetet ett enkelt exempel. Syftet är inte att bygga den mest avancerade GPT:n, utan att visa hela vägen från ett återkommande problem till en fungerande och förbättringsbar assistent.

Du kan följa samma steg med en egen idé medan du läser.

Så är boken upplagd

Vi börjar med vad en AI-assistent är och hur ett återkommande behov blir en tydlig GPT-idé. Därefter följer det praktiska huvudspåret: GPT Byggaren, utvecklingsplanen och den stegvisa utvecklingen i Chat-läge.

Först när du har sett processen i praktiken öppnar vi motorhuven och tittar på instruktioner, kunskap, förmågor och tester. Boken avslutas med förbättring, distribution och förvaltning.

Hur du får mest nytta av boken

Läs gärna boken med ChatGPT tillgängligt bredvid dig.

När ett kapitel innehåller ett praktiskt moment kan du prova det direkt. Du behöver inte använda exakt samma exempel som i boken. Det är ofta mer lärorikt att utgå från ett verkligt problem som du själv återkommer till.

Försök samtidigt att inte börja med en alltför stor idé. En första GPT blir lättare att förstå, testa och förbättra om den har ett tydligt uppdrag.

En bra utgångspunkt är därför inte:

Jag vill ha en AI som hjälper mig med allt i mitt arbete.

utan något i stil med:

Jag vill ha en GPT som analyserar den här typen av dokument och hjälper mig identifiera vad jag behöver agera på.

Det är där vi börjar.

1. Vad är egentligen en AI-assistent?

När du använder ChatGPT i en vanlig konversation möter du en generell AI. Den kan hjälpa till med många olika saker: förklara ett begrepp, sammanfatta en text, skriva ett utkast, analysera ett problem eller hjälpa dig med kod.

Det är en av styrkorna med ChatGPT. Du behöver inte bestämma i förväg exakt vilken sorts arbete du ska göra.

Men samma generalitet har också en baksida. Om du återkommer till samma arbetsuppgift behöver du ofta förklara samma saker igen:

- vilken roll AI:n ska ta,
- vilken typ av underlag den ska använda,
- vad den ska leta efter,
- hur resultatet ska struktureras,
- vad den ska undvika,
- vilka kvalitetskrav som gäller.

En specialiserad AI-assistent är ett sätt att göra mycket av detta återanvändbart.

Från generell AI till specialiserad assistent

Tänk dig att du regelbundet behöver läsa längre dokument och identifiera sådant som kan påverka din organisation.

Med vanlig ChatGPT kanske du varje gång skriver något i stil med:

Läs dokumentet. Identifiera sådant som kan påverka vår organisation. Sammanfatta de viktigaste punkterna, förklara varför de är relevanta och skilj tydligt mellan sådant som står i dokumentet och dina egna slutsatser.

Det kan fungera bra. Men nästa gång behöver du komma ihåg instruktionen igen. Kanske formulerar du den lite annorlunda och får ett annat resultat. En kollega kanske använder en helt annan instruktion.

En specialiserad dokumentanalys-GPT kan i stället redan vara utformad för uppgiften.

Du kan då ge den dokumentet och skriva något mycket enklare:

Analysera detta dokument.

Assistenten känner redan till sitt uppdrag, hur analysen ska göras och hur resultatet ska presenteras.

Det är den viktigaste skillnaden.

En vanlig ChatGPT-konversation formas huvudsakligen av det du säger just nu. En specialiserad GPT har dessutom ett återanvändbart arbetssätt som har utformats i förväg.

Du tränar normalt inte en ny AI-modell

Ordet GPT kan lätt ge intrycket att du skapar eller tränar en egen AI-modell. Det är normalt inte det som händer.

När vi i den här boken talar om att "bygga en GPT" menar vi att vi specialiserar hur en generell AI ska arbeta.

En förenklad mental modell är:

AI-modell + instruktioner + kunskap + verktyg + arbetsflöde = specialiserad AI-assistent

Alla assistenter behöver inte alla delarna. En enkel GPT kanske nästan bara består av bra instruktioner. En mer avancerad GPT kan behöva särskilt kunskapsmaterial, kunna läsa filer, söka på webben eller arbeta enligt ett tydligt flerstegsflöde.

Vi går igenom delarna närmare senare. Just nu räcker det att förstå vad de bidrar med.

Grundmodellen står för den generella förmågan

Den underliggande AI-modellen står för sådant som språkförståelse, resonemang och förmågan att skapa text.

Det är därför du inte behöver lära en dokumentanalys-GPT vad en sammanfattning är eller hur svenska meningar byggs upp. Den generella modellen har redan breda sådana förmågor.

Det du tillför är i stället specialiseringen.

Det kan jämföras med att anlita en erfaren generalist och ge personen ett tydligt uppdrag, arbetsinstruktioner, relevant material och rätt verktyg. Du behöver inte lära personen läsa från början. Du behöver få arbetet att utföras på rätt sätt i just din situation.

Instruktionerna beskriver hur assistenten ska arbeta

Instruktionerna kan till exempel ange att dokumentanalys-GPT:n ska:

- börja med att identifiera dokumentets syfte,
- skilja fakta från egna slutsatser,
- fokusera på konsekvenser för organisationen,
- markera osäkerheter,
- presentera resultatet kort och strukturerat.

Bra instruktioner handlar alltså inte bara om tonfall. De beskriver ofta **arbetsmetoden**.

Det är också här en specialiserad GPT blir mer konsekvent än en serie lösa promptar. Samma grundläggande arbetssätt kan användas gång på gång.

Knowledge ger assistenten relevant specialkunskap

I vissa användningsfall behöver assistenten känna till sådant som den generella modellen inte bör förväntas känna till tillräckligt väl.

Det kan exempelvis vara:

- interna begrepp,
- en metodbeskrivning,
- en produktkatalog,
- en klassificeringsmodell,
- organisationens riktlinjer,
- mallar för hur ett resultat ska se ut.

Sådant material kan ingå som **Knowledge**.

Det är viktigt att skilja Knowledge från instruktioner.

Instruktionen säger exempelvis:

Bedöm vilka verksamhetsområden som påverkas.

Knowledge kan innehålla:

Här är organisationens verksamhetsområden och hur de definieras.

Den ena delen beskriver **vad assistenten ska göra**. Den andra ger **material som hjälper den att göra det**.

Capabilities ger assistenten möjligheter

En assistent kan också behöva vissa funktionella förmågor, eller **capabilities**.

Om den bara ska diskutera text som du skriver i chatten krävs inte mycket mer än språkmodellen själv.

Men andra uppgifter kan kräva att assistenten kan:

- läsa bifogade filer,
- söka efter aktuell information på webben,
- analysera strukturerad data,
- skapa filer,
- använda andra tillgängliga verktyg.

Vilka capabilities som behövs beror alltså på arbetsuppgiften.

En vanlig fallgrop när man börjar tänka på GPT:er är att välja tekniska funktioner först och användningsfall sedan. Det är oftast bättre att göra tvärtom:

1. Vad behöver användaren få gjort?
2. Vilka underlag behövs?
3. Vilka funktioner krävs för att genomföra arbetet?

Det är också så vi kommer att arbeta med GPT Byggaren.

Arbetsflödet skapar struktur

Vissa uppgifter kan lösas i ett enda steg. Andra blir bättre om assistenten arbetar i en bestämd ordning.

Vår dokumentanalytiker skulle exempelvis kunna arbeta så här:

1. Identifiera vad dokumentet handlar om.
2. Hitta delar som kan vara relevanta för organisationen.
3. Bedöm möjliga konsekvenser.
4. Markera osäkerheter och informationsluckor.
5. Skapa en kort sammanställning.

Användaren behöver inte nödvändigtvis se dessa som fem separata steg. Men assistenten kan vara konstruerad så att arbetet blir mer systematiskt.

Det är en viktig del av specialisering: inte bara **vad AI:n vet**, utan **hur den arbetar**.

Samma fråga – olika förutsättningar

Anta att du bifogar ett nytt dokument och frågar:

Vad är viktigt för oss här?

En generell ChatGPT behöver först tolka vad "oss" betyder, vad du anser vara viktigt och hur svaret bör struktureras. Den kan förstå mycket från konversationen, men förutsättningarna är inte nödvändigtvis stabila från gång till gång.

Den specialiserade dokumentanalys-GPT:n kan redan ha följande förutsättningar:

- den vet vilken typ av organisation analysen gäller,
- den har ett definierat analysuppdrag,
- den vet vilka perspektiv den ska kontrollera,
- den följer en bestämd resultatstruktur,
- den markerar när underlaget inte räcker för en säker slutsats.

Frågan kan därför vara kort även om arbetsuppgiften bakom den är ganska avancerad.

Det är inte den korta prompten som gör assistenten bra. Det är att mycket av arbetssättet redan finns definierat.

Specialisering betyder inte ofelbarhet

En GPT blir inte automatiskt korrekt bara för att den är specialiserad.

Den kan fortfarande:

- misstolka ett dokument,
- dra en för stark slutsats,
- missa ett ovanligt fall,
- använda Knowledge på fel sätt,
- ge ett resultat som ser övertygande ut trots att underlaget är svagt.

Specialisering gör det däremot möjligt att ange ett bättre arbetssätt och sedan testa om assistenten följer det.

Det är därför vi längre fram kommer att behandla testning och förbättring som en naturlig del av utvecklingen, inte som något man gör först när något gått fel.

När är en specialiserad assistent värd att bygga?

Det finns ingen exakt gräns, men några signaler är särskilt användbara.

En specialiserad assistent är intressant när du märker att du:

- gör samma typ av uppgift återkommande,
- återanvänder ungefär samma instruktioner,
- vill ha resultat i en konsekvent form,
- behöver ge AI:n samma bakgrundsmaterial flera gånger,
- vill kunna dela arbetssättet med andra,
- vill kunna testa och förbättra beteendet över tid.

Däremot behöver varje fråga inte bli en egen GPT.

Om du bara ibland vill få hjälp att formulera ett mejl eller förstå ett begrepp är vanlig ChatGPT oftast enklare. Specialisering ger störst värde när **arbetssättet återkommer**.

En första modell att bära med sig

Du behöver ännu inte veta hur instruktioner, Knowledge, capabilities eller tester representeras i ett GPT-projekt.

Det viktiga efter det här kapitlet är att kunna se en AI-assistent som något mer konkret än ”en smart chatbot”.

Den är en generell AI som har fått ett särskilt uppdrag och ett mer stabilt sammanhang för hur uppdraget ska utföras.

Du kan därför tänka på konstruktionen i fyra frågor:

1. **Uppdrag:** Vad ska assistenten hjälpa till med?
2. **Arbetssätt:** Hur ska den angripa uppgiften?
3. **Underlag:** Vilken särskild kunskap eller information behöver den?
4. **Förmågor:** Vilka verktyg behöver den för att kunna göra jobbet?

Det är ungefär här GPT Byggaren kommer in. Du behöver inte själv översätta svaren på dessa frågor till en komplett teknisk GPT-struktur. I stället hjälper GPT Byggaren dig att analysera behovet och bygga projektet.

Men innan vi använder verktyget behöver vi kunna beskriva själva problemet tillräckligt tydligt.

Det är nästa steg.

Reflektion

Tänk på hur du själv använder ChatGPT i dag.

Finns det någon uppgift där du ofta behöver förklara ungefär samma sak innan ChatGPT kan börja hjälpa dig?

Skriv gärna ner tre saker:

- vilken uppgift du återkommer till,
- vilket underlag du brukar ge ChatGPT,
- hur du helst vill att resultatet ska se ut.

Du behöver inte formulera någon perfekt GPT-idé ännu. I nästa kapitel använder vi just den typen av observationer för att gå från ett löst behov till ett tydligt användningsfall.

Kort sammanfattning

En specialiserad GPT är normalt inte en nytränad AI-modell. Den bygger vidare på en generell AI men ger den ett tydligare och mer återanvändbart uppdrag.

Specialiseringen kan bestå av instruktioner, Knowledge, capabilities och ett definierat arbetsflöde. Det gör det möjligt att få ett mer konsekvent arbetssätt utan att behöva formulera hela uppgiften från början i varje konversation.

Nästa fråga är därför inte vilken teknik GPT:n ska använda, utan vilket problem den faktiskt ska lösa.

2. Från problem till GPT-idé

Det är lätt att börja i fel ände när man vill skapa en egen GPT.

Man börjar fundera på instruktioner, filer, webbsökning eller vilka verktyg assistenten behöver. Men innan något av det är viktigt behöver en enklare fråga vara besvarad:

Vilket återkommande problem ska assistenten hjälpa till att lösa?

En bra GPT börjar sällan med teknik. Den börjar med ett arbete som någon vill göra enklare, snabbare eller mer konsekvent.

I det här kapitlet går vi från ett löst behov till en tillräckligt tydlig GPT-idé för att GPT Byggaren ska kunna ta över mycket av resten.

Börja med arbetet, inte med GPT:n

Anta att du ofta får dokument som behöver läsas igenom för att avgöra om de påverkar din organisation.

En första idé kan vara:

Jag vill ha en GPT som analyserar dokument.

Det är en start, men fortfarande ganska otydligt. Nästan alla typer av dokument kan analyseras på många olika sätt.

En bättre beskrivning är:

Jag vill ha en GPT som hjälper mig läsa remisser och andra styrande dokument, identifierar sådant som kan påverka min organisation och sammanfattar de viktigaste konsekvenserna.

Nu börjar det bli möjligt att förstå uppgiften.

Det viktiga är inte att formuleringen är perfekt. Det viktiga är att den beskriver **arbetet och det önskade resultatet**.

Fem frågor räcker långt

För de flesta första GPT-idéer kommer du långt genom att svara på fem frågor.

1. Vem ska använda assistenten?

En GPT som är gjord för en jurist behöver kanske resonera annorlunda än en som är gjord för en chef eller en kundtjänstmedarbetare.

Frågan behöver inte besvaras med en exakt yrkestitel. Det kan räcka med:

- jag själv,
- personer i mitt team,
- handläggare,
- utvecklare,
- chefer,
- kunder.

För vårt exempel kan målgruppen vara:

Personer som behöver göra en första strukturerad bedömning av hur ett dokument påverkar organisationen.

2. Vilken uppgift ska den hjälpa till med?

Beskriv uppgiften som ett arbete, inte som en teknisk funktion.

Mindre bra:

GPT:n ska använda filer och skapa Markdown.

Bättre:

GPT:n ska läsa ett dokument, identifiera relevanta delar och sammanfatta vad organisationen behöver uppmärksamma.

Den senare formuleringen säger vad användaren vill få gjort. Hur det sedan implementeras kan GPT Byggaren hjälpa till att avgöra.

3. Vilket underlag får den?

Tänk på vad användaren faktiskt kommer att ge assistenten.

Det kan till exempel vara:

- ett PDF-dokument,
- text inklistrad i chatten,
- en webbsida,
- en kalkylfil,
- källkod,
- flera dokument som ska jämföras,
- interna riktlinjer som ska användas som referens.

I vårt exempel kan svaret vara:

Användaren bifogar ett dokument som ska analyseras. Assistenten kan också behöva känna till några fasta beskrivningar av organisationens uppdrag och ansvarsområden.

Den formuleringen börjar redan ge ledtrådar om att GPT:n kan behöva både filhantering och särskild Knowledge. Du behöver ändå inte bestämma den tekniska lösningen själv.

4. Hur ska resultatet se ut?

Det är ofta lättare att beskriva ett bra resultat än att beskriva hur assistenten ska arbeta.

Exempel:

Jag vill ha en kort sammanfattning, de viktigaste konsekvenserna, vilka områden som berörs och vilka frågor som behöver utredas vidare.

Detta är mycket mer användbart än att bara säga att GPT:n ska "analysera noggrant".

Ett tydligt önskat resultat hjälper både GPT Byggaren och den färdiga assistenten att förstå vad som faktiskt är viktigt.

5. Vad ska assistenten inte göra?

Avgränsningar är minst lika viktiga som funktioner.

I dokumentexemplet kan en viktig gräns vara:

Assistenten ska inte presentera osäkra slutsatser som fakta och ska inte fatta beslut åt användaren.

Andra vanliga gränser kan vara:

- inte lämna juridiska slutbedömningar,
- inte ändra filer utan uttryckligt uppdrag,
- inte använda externa källor när endast bifogat material ska bedömas,
- inte försöka göra flera olika arbetsuppgifter som egentligen borde vara separata.

Gränser gör GPT-idén tydligare och minskar risken att projektet växer åt alla håll.

Ett enkelt GPT-kort

Du behöver inte skriva en lång kravspecifikation. Ett litet "GPT-kort" räcker ofta för att komma vidare.

För vårt exempel kan det se ut så här:

Målgrupp: personer som gör en första bedömning av dokument som kan påverka organisationen.

Uppgift: läsa dokument och identifiera relevanta delar, konsekvenser och frågor för fortsatt analys.

Indata: främst bifogade dokument, ibland kompletterat med fast bakgrundskunskap om organisationen.

Utdata: en kort strukturerad sammanfattning med viktigaste påverkan, berörda områden och öppna frågor.

Gränser: inte fatta beslut åt användaren och inte framställa osäkra slutsatser som säkra fakta.

Det är redan tillräckligt bra för att börja arbeta med GPT Byggaren.

Du behöver inte lösa konstruktionen själv

När du har formulerat användningsfallet är det frestande att fortsätta med frågor som:

- Behöver jag Knowledge-filer?
- Ska den kunna söka på webben?
- Hur långa ska instruktionerna vara?
- Behöver jag schemas?
- Hur ska den testas?
- Ska den bli en Custom GPT eller användas som Chat ZIP?

Det är relevanta frågor, men de behöver inte vara dina första frågor.

GPT Byggaren är till för att hjälpa till med just den översättningen: från **vad du vill åstadkomma till hur GPT-projektet bör utformas**.

Du behöver därför framför allt kunna bedöma om GPT Byggaren har förstått verksamhetsbehovet rätt.

Det är en viktig arbetsfördelning:

Du beskriver problemet och bedömer om lösningen passar behovet. GPT Byggaren hjälper till att konstruera lösningen.

Undvik att göra GPT:n för bred

En vanlig första idé låter ungefär så här:

Jag vill ha en GPT som hjälper mig med allt i mitt arbete.

Det låter attraktivt, men ger ofta en sämre assistent.

Ju fler olika arbetsuppgifter som blandas ihop, desto svårare blir det att:

- ge tydliga instruktioner,
- veta vilket resultat som är rätt,
- välja relevant Knowledge,
- testa beteendet,
- förbättra assistenten när något fungerar dåligt.

En bättre start är ofta att välja **ett återkommande arbetsflöde**.

Exempelvis:

- analysera en remiss,
- granska ett kodprojekt,
- jämföra produkter,
- sammanfatta en viss typ av rapport,
- skapa ett första utkast enligt en bestämd struktur.

När den assistenten fungerar kan den utvecklas vidare, eller kompletteras med andra specialiserade assistenter.

Tänk i arbetsflöden

Ett bra sätt att pröva om en GPT-idé är tillräckligt konkret är att föreställa sig ett normalt användningstillfälle.

För dokumentanalysen kan arbetsflödet vara:

1. Användaren bifogar ett dokument.
2. Assistenten identifierar vilken typ av dokument det är.
3. Den läser efter sådant som är relevant för organisationen.
4. Den skiljer tydliga fakta från egna bedömningar.
5. Den sammanfattar påverkan i en återkommande struktur.
6. Den pekar ut sådant som behöver granskas vidare av en människa.

Det här är fortfarande inte en teknisk specifikation. Men det beskriver hur arbetet bör kännas för användaren.

Det är ofta precis lagom mycket information för nästa steg.

När är idén tillräckligt bra?

Du behöver inte vänta tills alla detaljer är lösta.

Din GPT-idé är vanligtvis tillräckligt tydlig när du kan beskriva:

- vem som ska använda den,
- vilken återkommande uppgift den ska hjälpa till med,
- vilket underlag den normalt får,
- vilket resultat användaren behöver,
- några viktiga gränser.

Om något fortfarande är oklart kan GPT Byggaren hjälpa dig upptäcka det under analysen.

Målet är alltså inte att skriva den perfekta prompten innan du börjar. Målet är att ge GPT Byggaren **ett begripligt problem att arbeta vidare med**.

Praktiskt moment: formulera din första idé

Ta uppgiften du funderade på i slutet av förra kapitlet och fyll i följande fem rader:

Målgrupp:

Uppgift:

Indata:

Utdata:

Gränser:

Försök hålla varje svar till en eller två meningar.

Skriv sedan en enkel startprompt utifrån dem. Den behöver inte innehålla alla detaljer ordagrant.

Till exempel:

Jag vill bygga en GPT som hjälper personer i min organisation att analysera remisser. Den ska läsa ett bifogat dokument, identifiera sådant som kan påverka organisationen och ge en kort strukturerad sammanfattning av konsekvenser och frågor som behöver utredas vidare. Den ska vara tydlig med osäkerheter och inte fatta beslut åt användaren.

Det är en fullt rimlig startpunkt.

Kort sammanfattning

En bra GPT-idé börjar med ett återkommande problem, inte med teknik.

Du kommer långt genom att beskriva fem saker: **målgrupp, uppgift, indata, utdata och gränser**. Det behöver inte bli en fullständig kravspecifikation.

När användningsfallet är tillräckligt tydligt kan GPT Byggaren hjälpa till att avgöra hur assistenten bör struktureras, vilken kunskap och vilka capabilities som behövs och hur projektet bör utvecklas.

I nästa kapitel tittar vi därför närmare på GPT Byggaren själv och på hur arbetsfördelningen mellan dig och byggverktyget fungerar.

3. GPT Byggaren

I förra kapitlet formulerade vi en GPT-idé utifrån målgrupp, uppgift, indata, utdata och gränser.

Det är ungefär där GPT Byggaren tar vid.

Tanken är att du inte ska behöva översätta verksamhetsbehovet till en fullständig teknisk konstruktion på egen hand. Du beskriver främst **vad assistenten ska hjälpa till med**. GPT Byggaren hjälper sedan till att avgöra **hur projektet bör byggas**.

Det betyder inte att du lämnar över alla beslut. Men arbetsfördelningen blir tydligare.

Du ansvarar för behovet och för att bedöma om resultatet blir användbart. GPT Byggaren ansvarar för mycket av konstruktionen, strukturen och kvalitetssäkringen.

Från idé till utvecklingsprojekt

Anta att du ger GPT Byggaren idén från förra kapitlet:

Jag vill bygga en GPT som hjälper personer i min organisation att analysera remisser. Den ska läsa ett bifogat dokument, identifiera sådant som kan påverka organisationen och ge en kort strukturerad sammanfattning av konsekvenser och frågor som behöver utredas vidare. Den ska vara tydlig med osäkerheter och inte fatta beslut åt användaren.

GPT Byggarens uppgift är då inte bara att skriva en lång instruktionstext.

Den behöver först förstå vad som faktiskt krävs.

Den kan exempelvis behöva bedöma:

- vilken målgrupp assistenten är gjord för,
- vilket arbetsflöde den ska följa,
- vilka typer av dokument den behöver kunna hantera,
- om särskild Knowledge behövs,
- om webbsökning eller andra capabilities behövs,
- hur resultatet ska struktureras,
- vilka risker eller gränser som behöver byggas in,
- hur beteendet bör testas,
- hur assistenten bör paketeras för användning.

Du behöver alltså inte börja med frågor som "ska jag ha ett schema?" eller "hur ska projektstrukturen se ut?".

GPT Byggaren kan ta många sådana tekniska beslut utifrån användningsfallet.

Vad behöver du själv bestämma?

Det finns ändå beslut som ett byggverktyg inte bör gissa sig till.

Om GPT Byggaren frågar:

Ska analysen vara en första orientering eller en formell juridisk bedömning?

är det ett verksamhetsbeslut. Svaret påverkar vad assistenten får göra och hur försiktigt den måste uttrycka sig.

Samma sak gäller frågor som:

- Vem är den egentliga användaren?
- Vad är ett godkänt resultat?
- Vilka källor får användas?
- Vilka uppgifter är känsliga?
- När ska assistenten stanna och be en människa ta över?
- Vad ska uttryckligen ligga utanför GPT:ns uppdrag?

Däremot behöver du normalt inte bestämma sådant som projektets interna filstruktur eller exakt hur en automatisk kontroll ska implementeras.

En användbar tumregel är:

Beslut om syfte, ansvar, kvalitet och gränser är dina. Beslut om teknisk realisering kan GPT Byggaren ofta hjälpa till med.

Först analys, sedan plan

GPT Byggaren arbetar stegvis.

Efter att du beskrivit vad du vill bygga analyserar den behovet. När det är tillräckligt tydligt tar den fram en utvecklingsplan anpassad till just projektet.

Planen kan exempelvis innehålla steg för att:

1. definiera assistentens uppdrag,
2. strukturera instruktionerna,
3. lägga till nödvändig Knowledge,
4. definiera viktiga arbetsflöden,
5. skapa tester och evals,
6. validera projektet,
7. bygga distributionspaket.

Det viktiga är att planen inte ska ses som en stel checklista.

Om ett test senare visar att något behöver rättas kan GPT Byggaren lägga in ett korrigeringssteg. Om ett planerat steg visar sig onödigt kan det hoppas över eller slås ihop med något annat.

Det gör utvecklingsplanen till en **riktning för arbetet**, inte ett kontrakt som måste följas mekaniskt.

Projekt-ZIP: utvecklingsprojektet

När GPT Byggaren börjar genomföra planen skapas normalt en **projekt-ZIP**.

Det är den viktigaste filen under själva utvecklingen.

Projekt-ZIP:en innehåller inte bara den färdiga GPT:n. Den innehåller hela utvecklingsprojektet: instruktioner, Knowledge, status, tester, dokumentation, byggstöd och andra filer som behövs för att fortsätta arbetet.

Du kan tänka på den som:

arbetsmappen för GPT-projektet, paketerad som en enda fil

Det är projekt-ZIP:en du sparar när du vill fortsätta utveckla assistenten senare.

Den gör också att projektet inte behöver vara beroende av en enda lång ChatGPT-konversation. Om du börjar i en ny konversation kan du bifoga den senaste projekt-ZIP:en och be GPT Byggaren fortsätta där projektet befinner sig.

Projektstatusen ligger alltså i själva projektet, inte bara i chathistoriken.

Chat ZIP: assistenten i en vanlig konversation

När projektet börjar bli användbart kan GPT Byggaren skapa en **Chat ZIP**.

Det är en runtime-version av assistenten som är avsedd att bifogas i en vanlig ChatGPT-konversation.

Arbetsmodellen är enkel:

1. Starta en ny konversation.
2. Bifoga Chat ZIP-filen.
3. Be ChatGPT använda ZIP-filen som den aktuella GPT-kontexten.
4. Börja använda den specialiserade assistenten.

Detta är bokens huvudspår.

En fördel är att samma grundidé som vi använder för GPT Byggaren också kan användas för GPT:n vi bygger: en ZIP-fil kan bära med sig instruktioner, Knowledge och annan struktur som gör en vanlig konversation till en mer specialiserad arbetsmiljö.

För dig som användare är det viktigaste att skilja Chat ZIP från projekt-ZIP:

Projekt-ZIP använder du för att **utveckla** GPT:n.

Chat ZIP använder du för att **köra** GPT:n i en konversation.

Custom GPT ZIP: material för GPT Builder

GPT Byggaren kan också skapa en **Custom GPT ZIP**.

Den innehåller material som är anpassat för att skapa eller uppdatera en Custom GPT i ChatGPT:s GPT Builder.

Det är alltså ytterligare ett sätt att distribuera samma grundläggande assistentidé.

En Custom GPT och en Chat ZIP behöver inte alltid vara tekniskt identiska. Plattformarna kan ha olika begränsningar för exempelvis hur mycket material som får plats eller hur olika funktioner kan användas.

GPT Byggaren försöker därför utgå från samma grundläggande beteende och sedan anpassa distributionen till respektive körsätt.

I den här boken behöver vi inte gå djupare än så ännu. Huvudspåret är Chat ZIP eftersom det gör hela kedjan enkel att förstå och prova.

Tre ZIP-filer – tre olika syften

De tre paketen är lätta att blanda ihop. Följande modell räcker långt:

Paket	Används till	Tänk på det som
Projekt-ZIP	fortsatt utveckling	hela arbetsprojektet
Chat ZIP	köra GPT:n i en vanlig ChatGPT-konversation	den körbara chatversionen
Custom GPT ZIP	skapa eller uppdatera en Custom GPT	distributionsunderlag för GPT Builder

För den praktiska processen i den här boken räcker det nästan att komma ihåg två saker:

Utveckla med projekt-ZIP. Använd i chatten med Chat ZIP.

Custom GPT återkommer kort senare när vi diskuterar hur en färdig assistent kan distribueras.

Det stegvisa arbetssättet

En viktig egenskap hos GPT Byggaren är att du inte behöver försöka skapa hela GPT:n i ett enda jättestort uppdrag.

Efter att utvecklingsplanen finns kan arbetet fortsätta stegvis.

En typisk instruktion kan vara:

Gör nästa steg och ge mig en uppdaterad zip.

GPT Byggaren läser då projektets aktuella status, avgör vilket steg som bör genomföras, gör arbetet, uppdaterar status och lämnar tillbaka en ny komplett projekt-ZIP.

Nästa gång utgår du från den nya filen.

Arbetssättet liknar därför mer vanlig iterativ utveckling än att försöka skriva ”den perfekta prompten” vid första försöket.

Det ger också en viktig trygghet: när en senare ändring behövs fortsätter du från ett faktiskt projekt i stället för att försöka återskapa allt från minnet.

GPT Byggaren ersätter inte ditt omdöme

Det kan vara lockande att tänka att byggverktyget nu löser hela problemet automatiskt.

Så fungerar det inte.

GPT Byggaren kan hjälpa till att skapa en välstrukturerad lösning, men du behöver fortfarande bedöma om lösningen passar det verkliga arbetet.

När du granskar resultatet är några enkla frågor viktigare än att förstå varje teknisk fil:

- Har GPT Byggaren förstått uppgiften rätt?
- Är målgruppen rätt beskriven?
- Saknas något viktigt underlag?
- Är resultatet tänkt att se ut på rätt sätt?
- Finns de viktigaste gränserna med?
- Verkar planen bygga rätt assistent, inte bara en tekniskt avancerad assistent?

Det är här din verksamhetskunskap har störst värde.

En mental modell för hela processen

Nu kan vi sätta ihop flödet från förra kapitlet med GPT Byggaren:

**Problem → GPT-idé → analys → utvecklingsplan → projekt-ZIP → stegvis utveckling
→ testad GPT → Chat ZIP eller Custom GPT**

Det är i praktiken bokens huvudflöde.

Du behöver förstå vad som händer i varje del, men du behöver inte manuellt konstruera varje teknisk komponent.

I nästa kapitel gör vi detta på riktigt. Då startar vi GPT Byggaren i Chat-läge, beskriver en GPT-idé, granskar planen och börjar bygga den första versionen.

Reflektion

Titta på GPT-kortet du gjorde i förra kapitlet.

Markera vilka delar som tydligt är **dina verksamhetsbeslut**. Det kan exempelvis vara målgrupp, syfte, kvalitetskrav och gränser.

Fundera sedan på vilka frågor du gärna skulle låta GPT Byggaren hjälpa dig med. Det kan exempelvis vara hur instruktionerna ska struktureras, vilken Knowledge som behövs och hur assistenten bör testas.

Om uppdelningen känns tydlig har du förstått den viktigaste idén med GPT Byggaren.

Kort sammanfattning

GPT Byggaren hjälper dig gå från en beskriven GPT-idé till ett strukturerat, testbart och distribuerbart GPT-projekt. Du ansvarar för behovet och bedömningen av nyttan; GPT Byggaren tar ett större ansvar för den tekniska realiseringen.

Nästa steg är att använda arbetsflödet praktiskt och bygga den första GPT:n.

4. Bygg din första GPT

Nu har vi tillräckligt med bakgrund för att göra det praktiskt.

I det här kapitlet använder vi GPT Byggaren i **Chat-läge** för att gå från en idé till ett faktiskt GPT-projekt. Du behöver inte förstå projektets alla tekniska delar. Fokus ligger på arbetsflödet: vad du gör, vad GPT Byggaren gör och vad du behöver kontrollera på vägen.

Vi fortsätter med samma exempel som tidigare: en GPT som hjälper användaren analysera dokument och lyfta fram sådant som kan påverka organisationen.

Steg 1 – Starta GPT Byggaren

Börja med att hämta den senaste **Chat ZIP-versionen** av GPT Byggaren från projektets GitHub Releases. Filnamnet följer normalt mönstret `gpt-byggaren-chat-<version>.zip`.

Starta sedan en ny ChatGPT-konversation och bifoga ZIP-filen.

Skriv exempelvis:

Använd denna zip som GPT i denna konversation.

ChatGPT får då tillgång till GPT Byggarens instruktioner och det övriga material som behövs för att använda verktyget.

Det är en viktig detalj: du installerar inte ett program på datorn. Du ger i stället den aktuella ChatGPT-konversationen den kontext som behövs för att agera som GPT Byggaren.

När detta är gjort kan du börja beskriva den GPT du själv vill skapa.

Steg 2 – Beskriv vad du vill bygga

Du behöver inte börja med en teknisk specifikation.

För vårt exempel kan startprompten vara:

Jag vill bygga en GPT som hjälper personer i min organisation att analysera remisser och andra dokument. Den ska läsa ett bifogat dokument, identifiera sådant som kan påverka organisationen och ge en kort strukturerad sammanfattning av konsekvenser och frågor som behöver utredas vidare. Den ska vara tydlig med osäkerheter och inte fatta beslut åt användaren.

Detta räcker långt.

Du har beskrivit:

- vem assistenten är till för,
- vilken uppgift den ska hjälpa till med,
- vilken typ av indata den får,
- vilket resultat du vill ha,
- en viktig gräns.

GPT Byggaren kan sedan analysera vad som behövs för att göra idén till en fungerande assistent.

Steg 3 – Svara på verksamhetsfrågor, inte teknikfrågor

GPT Byggaren kan ibland behöva ställa följdfrågor.

Det viktiga är att skilja mellan två typer av frågor.

Den första typen gäller verksamheten:

- Ska GPT:n ge en orienterande analys eller en formell bedömning?
- Vilka dokumenttyper ska omfattas?
- Vilka användare ska resultatet vara begripligt för?
- Vilka källor får användas?
- Vad får GPT:n absolut inte göra?

Sådana frågor behöver du besvara.

Den andra typen gäller teknisk konstruktion:

- Hur ska projektets interna struktur se ut?
- Behövs schemas?
- Hur ska tester organiseras?
- Hur ska distributionspaketen byggas?

Sådant ska GPT Byggaren normalt hjälpa till att avgöra.

Du behöver alltså inte bli teknisk projektledare bara för att du använder GPT Byggaren.

Steg 4 – Granska analysen

Innan utvecklingen börjar bör GPT Byggaren sammanfatta hur den har förstått behovet.

Läs denna del noggrant.

Kontrollera framför allt:

- Har den förstått rätt användare?
- Har den förstått huvuduppgiften?
- Har den lagt till funktioner du inte behöver?
- Saknas någon viktig begränsning?
- Är resultatet tänkt att bli användbart i det verkliga arbetet?

Det är lätt att fastna i tekniska formuleringar, men den viktigaste kvalitetskontrollen är mycket enklare:

Bygger planen rätt assistent för rätt problem?

Om svaret är nej bör du korrigera förståelsen innan utvecklingen går vidare.

Steg 5 – Granska utvecklingsplanen

När behovet är tillräckligt tydligt skapar GPT Byggaren en utvecklingsplan.

Planen kan till exempel innehålla steg för att:

1. definiera assistentens uppdrag,
2. skapa grundinstruktioner,
3. lägga till nödvändig Knowledge,
4. definiera arbetsflödet för dokumentanalys,
5. skapa tester och evals,
6. validera projektet,
7. bygga Chat ZIP och andra distributioner.

Du behöver inte bedöma om varje tekniskt steg är exakt rätt utfört. Däremot bör du kontrollera att planen verkar leda mot den assistent du faktiskt vill ha.

Fråga dig:

- Täcker planen huvuduppgiften?
- Finns testning med?
- Finns de viktigaste riskerna och gränserna med?
- Verkar projektet onödigt avancerat?
- Saknas något som användaren verkligen behöver?

Om planen ser rimlig ut kan utvecklingen börja.

Steg 6 – Skapa första projekt-ZIP:en

Be GPT Byggaren genomföra första steget.

Du kan skriva:

Gör första steget och ge mig resultatet som en zip.

När projektet har skapats får du en **projekt-ZIP**.

Spara den.

Det här är nu den viktigaste filen i utvecklingsarbetet. Projekt-ZIP:en innehåller projektets aktuella tillstånd och används som grund för nästa utvecklingssteg.

Du behöver normalt inte packa upp ZIP-filen och börja redigera dess innehåll manuellt. Den viktigaste uppgiften för dig är att hålla reda på den senaste versionen.

Steg 7 – Fortsätt med nästa steg

När du vill fortsätta kan instruktionen vara mycket enkel:

Gör nästa steg och ge mig en uppdaterad zip.

GPT Byggaren ska då läsa projektets aktuella status och avgöra vilket arbete som är nästa rimliga steg.

Efter genomfört arbete får du en ny projekt-ZIP.

Arbetsmönstret blir därför:

senaste projekt-ZIP → nästa steg → kontroll → ny projekt-ZIP

Du behöver inte själv hålla reda på att exempelvis ”steg 7 måste följas av steg 8”. Om ett test visar ett problem kan GPT Byggaren behöva göra ett korrigeringssteg först.

Det är en av styrkorna med att projektstatus finns i själva projektet.

Steg 8 – Läs vad som faktiskt förändrades

Även om du inte behöver förstå varje teknisk fil bör du läsa sammanfattningen efter varje större steg.

Titta särskilt efter:

- vad som har lagts till,
- vad som har ändrats,
- vilka antaganden som gjorts,
- om något fortfarande är öppet,
- vilket nästa steg rekommenderas.

Det gör att du kan ingripa tidigt om projektet börjar utvecklas i fel riktning.

Ett bra arbetssätt är att inte bara svara ”fortsätt” reflexmässigt. När något viktigt för verksamheten förändras bör du bedöma om det fortfarande stämmer med ditt mål.

Steg 9 – Prova den första användbara versionen

Efter några utvecklingssteg kommer projektet till en punkt där det går att prova assistenten som användare.

Då är det dags att bygga eller använda en **Chat ZIP** för den GPT som projektet skapar.

Starta en ny ChatGPT-konversation, bifoga Chat ZIP-filen och be ChatGPT använda den som GPT-kontext.

Testa sedan med ett realistiskt dokument.

För vårt exempel kan du kontrollera:

- hittar assistenten de viktigaste delarna?
- sammanfattar den på rätt detaljnivå?
- skiljer den fakta från bedömning?
- lyfter den osäkerheter?
- blir resultatet faktiskt användbart för målgruppen?

Detta är viktigare än att bara fråga om projektets automatiska tester är gröna.

Automatiska tester kan kontrollera många egenskaper, men bara en verklig användare kan avgöra om assistenten hjälper till med det verkliga arbetet.

Steg 10 – Ge återkoppling som beskriver problemet

Om något inte fungerar behöver du inte alltid tala om exakt hur det ska lösas.

Skriv vad du observerar.

Exempel:

Analysen blir för lång. Jag vill att den viktigaste påverkan ska komma först och att detaljer bara tas med när de behövs för att förstå slutsatsen.

Eller:

GPT:n verkar ibland anta att en formulering i dokumentet påverkar organisationen trots att kopplingen är osäker. Jag vill att sådana fall markeras tydligt som något som behöver verifieras.

Detta är ofta bättre än att själv försöka skriva om de tekniska instruktionerna.

Du beskriver **beteendet som behöver förbättras**. GPT Byggaren kan sedan avgöra vilka delar av projektet som behöver ändras och vilka tester som bör uppdateras.

Steg 11 – Fortsätt i en ny konversation

En lång utvecklingskonversation behöver inte leva för alltid.

När du vill fortsätta i en ny konversation gör du i princip samma sak som tidigare:

1. Starta en ny konversation.
2. Bifoga GPT Byggarens Chat ZIP.
3. Bifoga den senaste projekt-ZIP:en för GPT:n du utvecklar.
4. Be GPT Byggaren fortsätta projektet.

Exempel:

Fortsätt utveckla detta GPT-projekt och gör nästa rekommenderade steg.

Eftersom projektstatus och viktiga beslut finns i projektet behöver du normalt inte återberätta hela historiken.

Det är därför viktigt att alltid utgå från **senaste projekt-ZIP:en**.

Ett komplett arbetsflöde

Hela processen kan sammanfattas så här:

1. Hämta GPT Byggarens Chat ZIP.
2. Starta GPT Byggaren i en ny konversation.
3. Beskriv vilket problem din GPT ska lösa.
4. Besvara eventuella verksamhetsfrågor.
5. Granska GPT Byggarens analys.
6. Granska utvecklingsplanen.
7. Be GPT Byggaren skapa första projekt-ZIP:en.
8. Fortsätt stegvis med den senaste projekt-ZIP:en.
9. Prova den framväxande GPT:n med realistiska exempel.
10. Beskriv problem och förbättringsbehov.
11. Fortsätt tills assistenten är tillräckligt bra för sitt syfte.
12. Bygg den distribution du vill använda.

Det kan se ut som många steg när de listas så här, men i praktiken består arbetet till stor del av en konversation.

Du beskriver vad du vill åstadkomma, granskar viktiga beslut och ber sedan GPT Byggaren fortsätta.

Ett misstag att undvika: bygg allt innan du provar

Det är frestande att låta hela utvecklingsplanen köras färdigt innan du testar något.

Det är sällan det bästa arbetssättet.

Om GPT Byggaren har missförstått en central del av användningsfallet är det bättre att upptäcka det efter några steg än efter hela projektet.

Prova därför assistenten när det finns en första meningsfull version.

Frågan är inte:

Är GPT:n färdig?

utan snarare:

Har vi byggt tillräckligt mycket för att lära oss något genom att använda den?

Det gör utvecklingen både snabbare och mer träffsäker.

Praktiskt moment: starta ditt projekt

Om du följer boken praktiskt är det nu dags att skapa ditt eget projekt.

Utgå från GPT-kortet från kapitel 2.

Starta GPT Byggaren i Chat-läge och ge den en kort beskrivning av din idé.

När analysen kommer tillbaka, kontrollera tre saker innan du går vidare:

1. Problemförståelse

Har GPT Byggaren förstått vilket arbete som ska underlättas?

2. Resultat

Har den förstått vad användaren behöver få tillbaka?

3. Gränser

Har de viktigaste begränsningarna följt med?

Om svaret är ja kan du låta GPT Byggaren skapa utvecklingsplanen och därefter första projekt-ZIP:en.

Spara den senaste ZIP-filen. Den är startpunkten för resten av arbetet.

Kort sammanfattning

Att bygga en GPT med GPT Byggaren är i första hand ett **iterativt samtal om behov och kvalitet**, inte en övning i att manuellt skriva alla tekniska komponenter.

Du startar GPT Byggaren med dess Chat ZIP, beskriver din GPT-idé och granskar hur behovet har förståtts. Därefter skapas en utvecklingsplan och en projekt-ZIP som utvecklas steg för steg.

Den viktigaste arbetsrytmen är enkel:

Beskriv behovet → granska → bygg ett steg → prova → förbättra → fortsätt.

I nästa kapitel tittar vi på vad GPT Byggaren faktiskt skapar inne i projektet. Målet är inte att du ska börja redigera allt manuellt, utan att du ska förstå de viktigaste byggstenarna och varför de finns.

5. Vad GPT Byggaren bygger åt dig

I förra kapitlet byggde vi en första GPT utan att behöva förstå projektets alla tekniska delar. Det är en av poängerna med GPT Byggaren: du ska kunna börja i verksamhetsbehovet och låta verktyget ta ansvar för en stor del av konstruktionen.

Men det är ändå värdefullt att förstå **vad som faktiskt byggs**.

Inte för att du ska börja redigera varje fil manuellt, utan för att du ska kunna bedöma om assistenten har rätt beteende, rätt kunskap och rätt gränser.

Ett enkelt sätt att se på en GPT är att den består av flera delar:

uppdrag + instruktioner + kunskap + förmågor + arbetsflöden + kvalitetskontroller + distribution

GPT Byggaren hjälper dig att omsätta din idé till dessa delar och hålla dem samordnade.

Instruktionerna styr hur assistenten arbetar

Instruktionerna beskriver hur GPT:n ska bete sig.

För vår dokumentanalys-GPT kan instruktionerna till exempel säga att den ska:

- identifiera sådant som kan påverka organisationen,
- skilja fakta i dokumentet från egna slutsatser,
- markera osäkerheter,
- prioritera viktig påverkan framför detaljer,
- inte fatta beslut åt användaren.

Detta är mer än en startprompt.

Instruktionerna fungerar som assistentens återkommande arbetssätt. När en ny användare startar en ny konversation ska samma grundläggande beteende fortfarande gälla.

Det är därför en bra GPT inte bara är en samling smarta formuleringar. Instruktionerna behöver tillsammans bilda ett begripligt och konsekvent arbetssätt.

Beteende är inte samma sak som kunskap

En viktig skillnad är mellan **hur GPT:n ska arbeta** och **vad den behöver känna till**.

Anta att dokumentanalys-GPT:n ska analysera material utifrån en organisations interna principer.

Då kan följande vara en instruktion:

Jämför dokumentets innehåll med organisationens styrande principer och lyft möjliga konflikter.

Själva principerna hör däremot normalt hemma som kunskapsunderlag.

Det ger en enkel tumregel:

Instruktioner beskriver beteendet. Knowledge beskriver sådant assistenten behöver veta.

Gränsen är inte alltid perfekt, men den är mycket användbar när du bedömer ett projekt.

Knowledge ger assistenten specialiserad kunskap

En generell språkmodell kan mycket, men den känner inte automatiskt till allt som är specifikt för din organisation, metod eller uppgift.

Därför kan en GPT få särskilt **Knowledge-material**.

Det kan exempelvis vara:

- interna riktlinjer,
- metodbeskrivningar,
- begreppslistor,
- mallar,
- exempel på önskat resultat,
- offentliga regelverk eller andra stabila referensdokument.

I dokumentanalys-exemplet skulle vi kunna ge assistenten en kort beskrivning av organisationens uppdrag, vilka områden den ansvarar för och vilka typer av konsekvenser som är särskilt viktiga att uppmärksamma.

Då behöver vi inte pressa in all denna information i huvudinstruktionen.

Knowledge ska ha ett tydligt syfte

Mer material är inte automatiskt bättre.

Om du lägger in stora mängder dokument utan tydligt syfte blir det svårare att förstå vilken information GPT:n faktiskt förväntas använda.

Fråga därför för varje kunskapskälla:

- Vad ska GPT:n använda denna information till?
- I vilka situationer är den relevant?
- Är informationen stabil nog att ligga i projektet?
- Riskerar den att bli inaktuell?

GPT Byggaren kan hjälpa till att strukturera materialet, men du behöver kunna bedöma om kunskapen verkligen är rätt för verksamhetsuppgiften.

Capabilities avgör vad assistenten kan göra

En GPT behöver ibland mer än bara instruktioner och statisk kunskap.

Den kan exempelvis behöva kunna:

- läsa bifogade filer,
- söka på webben,
- analysera strukturerad information,
- skapa filer,
- köra beräkningar eller scripts,
- använda andra tillgängliga verktyg.

I GPT-projekt kallas sådana förmågor ofta **capabilities**.

För dokumentanalys-GPT:n är filhantering central. Om användaren inte kan ge assistenten dokumentet som ska analyseras faller hela användningsfallet.

En annan GPT kanske främst behöver webbsökning. En tredje behöver ingen extern förmåga alls.

Det viktiga är därför inte att aktivera så mycket som möjligt, utan att fråga:

Vilka förmågor krävs för att lösa den faktiska uppgiften?

GPT Byggaren ska hjälpa dig göra denna bedömning från användningsfallet i stället för att du först

behöver lära dig alla tekniska alternativ.

Arbetsflödet binder ihop delarna

Instruktioner, Knowledge och capabilities räcker inte alltid var för sig.

För mer avancerade uppgifter behöver assistenten också ett tydligt **arbetsflöde**.

För dokumentanalys-GPT:n kan det förenklat vara:

1. förstå vad dokumentet handlar om,
2. identifiera relevanta delar,
3. bedöma möjlig påverkan,
4. skilja säkra observationer från osäkerheter,
5. prioritera det viktigaste,
6. presentera resultatet i en återkommande struktur.

Ett sådant arbetsflöde gör beteendet mer förutsägbart.

Det betyder inte att GPT:n måste följa ett stelt program där varje fråga alltid behandlas exakt likadant. Men den får en tydligare metod att utgå från.

Det är särskilt värdefullt när uppgiften består av flera moment och när resultatet ska bli konsekvent mellan olika användningar.

Tester kontrollerar sådant som går att kontrollera automatiskt

När GPT Byggaren skapar ett riktigt projekt kan det också innehålla tester.

Det kan först kännas överdrivet. En GPT är ju inte ett traditionellt program där samma indata alltid måste ge exakt samma text.

Men mycket går ändå att kontrollera.

Tester kan exempelvis upptäcka att:

- en obligatorisk projektfil saknas,
- en konfigurationsfil har fel format,
- ett script eller en transformation ger fel resultat,
- en distribution inte går att bygga eller saknar obligatoriska filer,
- en distribution inte kan starta med sina viktigaste resurser,
- ett förväntat projektkontrakt inte längre uppfylls.

Grundprincipen är enkel: **sådant som har ett exakt kontrollerbart svar bör testas deterministiskt**. Det kan gälla projektstruktur, scheman, byggsteg, distribution och vissa egenskaper i körningen.

De kontrollerna gör att tekniska och strukturella fel kan upptäckas tidigt i stället för först när någon försöker använda eller distribuera GPT:n.

Evals provar assistentens beteende

För en AI-assistent räcker strukturella tester inte hela vägen.

Vi vill också veta hur GPT:n beter sig i realistiska situationer.

Där kommer **evals** in.

En eval är förenklat ett scenario där vi provar om assistenten beter sig som avsett.

För vårt exempel skulle scenarier kunna vara:

- ett dokument med tydlig påverkan på organisationen,
- ett dokument där påverkan är osäker,
- ett dokument som i praktiken inte berör organisationen,
- ett dokument där användaren försöker få GPT:n att dra en säkrare slutsats än underlaget medger.

Det viktiga är inte att svaret har exakt samma formulering varje gång.

Det viktiga är att centrala egenskaper håller:

- relevant påverkan identifieras,
- osäkerhet markeras,
- irrelevant innehåll får inte dominera,
- assistenten överskrider inte sin roll.

Automatisk kvalitet och mänsklig kvalitet kompletterar varandra

Tester och evals är värdefulla, men de kan inte ensamma avgöra om en GPT är bra.

En assistent kan passera alla automatiska kontroller och ändå vara frustrerande att använda.

Därför behöver du fortfarande prova den med verkliga eller realistiska uppgifter.

Tänk på kvalitet i två lager:

**Automatiska kontroller skyddar projektet mot kända fel.
Mänsklig användning visar om assistenten faktiskt är hjälpsam.**

Det ena ersätter inte det andra.

Projektstatus håller ihop utvecklingen

Ett GPT-projekt behöver också veta **var i utvecklingen det befinner sig**. Därför lagrar GPT Byggaren projektets syfte, viktiga beslut, genomförda steg och nästa rekommenderade arbete i projektet.

Det är det som gör projekt-ZIP:en återupptagningsbar. När du fortsätter senare behöver GPT Byggaren inte förlita sig på att hela den gamla chatthistoriken finns kvar.

När assistenten ska användas byggs i stället en distribution, exempelvis Chat ZIP eller Custom GPT. Skillnaden mellan dessa gick vi igenom tidigare; här räcker det att komma ihåg att **projektet är utvecklingsunderlaget och distributionen är det användaren kör**.

Från en önskan till flera projektdelar

Nu kan vi följa ett konkret önskemål genom projektet.

Anta att du säger:

Jag vill att GPT:n alltid markerar när den är osäker på om dokumentet faktiskt påverkar organisationen.

Det kan påverka flera delar samtidigt.

Instruktionen kan behöva beskriva hur osäkerheten ska hanteras.

Arbetsflödet kan behöva lägga in ett uttryckligt steg där säkerheten i bedömningen värderas.

Resultatformatet kan behöva få en markering för osäker påverkan.

Evals kan behöva innehålla ett scenario där kopplingen till organisationen är oklar.

Det är här GPT Byggaren gör stor nytta: ett verksamhetskrav behöver ofta bli flera samordnade delar i projektet.

Som användare behöver du inte själv känna till exakt vilka filer som ska ändras.

Men du bör kunna kontrollera att resultatet motsvarar önskemålet.

Vad behöver du egentligen förstå?

Efter detta kapitel behöver du inte kunna bygga projektstrukturen för hand.

Det viktiga är att du kan skilja på följande:

Del	Frågan den besvarar
Uppdrag	Vad ska assistenten hjälpa till med?
Instruktioner	Hur ska den bete sig?
Knowledge	Vad behöver den känna till?
Capabilities	Vad behöver den kunna göra?
Arbetsflöde	Hur ska en mer sammansatt uppgift genomföras?
Tester	Är projektets struktur och kontrakt fortfarande hela?
Evals	Beter sig assistenten rätt i viktiga scenarier?
Projektstatus	Var befinner sig utvecklingen och vad händer härnäst?
Distribution	Hur paketeras assistenten för faktisk användning?

Den modellen räcker långt när du granskar vad GPT Byggaren producerar.

Praktisk reflektion: beteende eller kunskap?

Tänk på din egen GPT-idé från kapitel 2.

Skriv ner två saker:

1. Något GPT:n **alltid ska göra på ett visst sätt**.
2. Något GPT:n **behöver känna till för att kunna göra ett bra jobb**.

Den första punkten hör sannolikt hemma i beteendet och instruktionerna.

Den andra hör sannolikt hemma i Knowledge eller i en informationskälla som GPT:n kan använda.

Exempel från vår dokumentanalys-GPT:

Beteende:

Markera tydligt när kopplingen mellan dokumentets innehåll och organisationen är osäker.

Kunskap:

Organisationens uppdrag, ansvarsområden och centrala styrande principer.

Den enkla skillnaden hjälper dig upptäcka många designproblem tidigt.

När behöver du öppna motorhuven?

Ibland behöver du titta djupare i projektet.

Det kan vara motiverat när:

- GPT:n beter sig fel trots flera vanliga förbättringsförsök,
- samma problem återkommer efter ändringar,
- du behöver förstå exakt vilken kunskap en slutsats bygger på,
- en distribution fungerar annorlunda än en annan,
- tester eller validering rapporterar ett tekniskt problem,
- projektet ska lämnas över till någon som ska förvalta det.

Men börja inte där.

För de flesta ändringar är det bättre att först beskriva **vilket beteende som är fel eller vilket behov som saknas** och låta GPT Byggaren avgöra vilka interna delar som behöver ändras.

Det håller fokus på problemet i stället för implementationen.

Sammanfattning

GPT Byggaren skapar mer än en lång prompt.

Den hjälper till att bygga ett sammanhängande GPT-projekt där:

- instruktioner styr beteendet,
- Knowledge ger specialiserad kunskap,
- capabilities ger nödvändiga förmågor,
- arbetsflöden strukturerar sammansatta uppgifter,
- tester skyddar projektets tekniska kontrakt,
- evals provar viktiga beteenden,
- projektstatus gör utvecklingen återupptagningsbar,
- distributioner gör assistenten användbar i rätt miljö.

Du behöver förstå vad delarna **är till för**, men du behöver inte konstruera dem manuellt.

I nästa kapitel använder vi denna förståelse för något viktigare: att **testa assistenten i realistiska situationer och förbättra den utan att börja om från början**.

6. Testa, förbättra och fortsatt utveckla

När den första versionen av din GPT finns är det lätt att tänka att projektet nästan är klart.

I praktiken börjar då en av de viktigaste delarna av arbetet: att **prova assistenten i verkliga situationer och förbättra den utifrån det du ser**.

En GPT är inte som ett vanligt formulär där alla möjliga indata kan listas i förväg. Användare formulerar sig olika, dokument ser olika ut och situationer innehåller ofta sådant du inte tänkte på när idén först beskrevs.

Därför är det bättre att se den första fungerande versionen som början på en förbättringscykel:

bygg → prova → observera → förbättra → prova igen

GPT Byggaren hjälper till med själva förändringsarbetet, men du behöver fortfarande avgöra om assistenten fungerar bra i den verklighet där den ska användas.

Börja med realistiska uppgifter

Det enklaste sättet att testa en GPT är att använda den så som en riktig användare skulle göra.

Om du har byggt en dokumentanalys-GPT bör du alltså inte börja med konstruerade frågor som bara kontrollerar om en viss instruktion finns. Ge den i stället några dokument som liknar sådant den faktiskt kommer att möta.

Välj gärna olika typer av situationer:

- ett enkelt fall där rätt svar är ganska tydligt,
- ett mer omfattande dokument,
- ett fall där påverkan är osäker,
- ett fall där nästan inget är relevant,
- ett fall där information saknas,
- en fråga där användaren uttrycker sig otydligt.

Du försöker inte bevisa att GPT:n fungerar perfekt. Du försöker upptäcka **var den inte fungerar tillräckligt bra ännu**.

Bedöm mer än om svaret är korrekt

När du provar assistenten är det frestande att bara fråga: "Blev svaret rätt?"

Det är viktigt, men ofta inte tillräckligt.

En användbar GPT behöver också fungera bra som arbetsverktyg.

Fundera därför på exempelvis:

- Hittar den det viktigaste?
- Missar den något som en människa normalt skulle reagera på?
- Läger den för mycket vikt vid oviktiga detaljer?
- Skiljer den fakta från egna slutsatser?
- Markerar den osäkerhet på ett begripligt sätt?
- Är resultatet lagom långt?
- Är strukturen konsekvent mellan olika körningar?
- Ställer den frågor när information verkligen saknas?
- Undviker den onödiga följdfrågor?
- Håller den sig inom sitt uppdrag?

Detta är ofta mer värdefullt än att försöka bedöma varje enskild formulering.

Beskriv problemet, inte den tekniska lösningen

När du hittar ett problem kan du återvända till GPT Byggaren och beskriva vad som inte fungerar.

Anta att dokumentanalys-GPT:n ger mycket långa svar där små detaljer får lika stor plats som viktiga konsekvenser.

Du behöver inte säga:

Ändra instruktion 4.2 och lägg till en prioriteringsregel före output-schemat.

Det räcker bättre att beskriva observationen och önskat beteende:

När jag provar GPT:n på längre dokument blir svaren för omfattande. Viktiga konsekvenser försvinner bland detaljer. Jag vill att den prioriterar de viktigaste konsekvenserna först och håller övriga observationer kortare.

Det ger GPT Byggaren möjlighet att avgöra **vilka delar av projektet som behöver ändras**.

Det kan handla om instruktioner, arbetsflöde, exempel, evals eller flera delar samtidigt.

Samma princip som när projektet startades gäller alltså fortfarande:

Beskriv verksamhetsproblemet så tydligt som möjligt. Låt GPT Byggaren föreslå hur projektet bör förändras.

Bra återkoppling är konkret

Jämför två sätt att beskriva ett problem.

Det första är svårt att arbeta vidare från:

Resultatet känns inte riktigt bra.

Det andra ger betydligt bättre underlag:

När dokumentet innehåller både ekonomiska och organisatoriska konsekvenser beskriver GPT:n ekonomin utförligt men missar ofta behovet av förändrade arbetssätt. Jag vill att den alltid bedömer båda typerna när underlaget ger stöd för det.

En användbar återkoppling innehåller ofta tre delar:

1. **Situation** – när händer problemet?
2. **Observation** – vad gör GPT:n idag?
3. **Önskat beteende** – hur borde den agera i stället?

Du behöver inte skriva detta som ett formellt formulär. Modellen är bara ett sätt att göra problemet tydligt.

Ändra en sak utan att förstöra något annat

En förbättring kan ibland skapa ett nytt problem.

Anta att vi ber GPT:n att alltid vara mycket kortfattad. Det kanske löser problemet med långa svar, men samtidigt gör att viktiga reservationer eller förklaringar försvinner.

Det är därför man behöver **regressionstestning**.

Ordet låter tekniskt, men principen är enkel:

När du har förbättrat något ska du också kontrollera att sådant som redan fungerade fortfarande fungerar.

Om du exempelvis förbättrar prioriteringen av viktiga konsekvenser kan du köra några tidigare testfall igen och kontrollera att GPT:n fortfarande:

- markerar osäkerheter,
- skiljer fakta från slutsatser,
- ignorerar irrelevant material,
- inte börjar fatta beslut åt användaren.

GPT Byggaren kan lägga till eller uppdatera tester och evals när förändringen motiverar det. Du behöver inte själv implementera testsystemet för att dra nytta av principen.

Korrigeringssteg är en normal del av processen

En utvecklingsplan är inte en checklista som måste följas mekaniskt från första till sista raden.

Under arbetet kan exempelvis en validering visa att något behöver rättas innan nästa planerade steg är meningsfullt.

Då kan flödet se ut så här:

```
planerat steg
↓
kontroll hittar ett problem
↓
korrigeringssteg
↓
ny kontroll
↓
fortsatt utveckling
```

Detta är inte ett misslyckande. Det är snarare ett tecken på att projektet använder sina kvalitetskontroller.

På samma sätt kan GPT Byggaren upptäcka att två planerade steg bör slås ihop eller att ett steg inte längre behövs.

Det viktiga är projektets faktiska tillstånd, inte att stegnumren följs blint.

Spara den version du faktiskt förbättrar

Efter varje förbättringsvarv bör du spara den senaste projekt-ZIP:en. Den innehåller både projektet och den status GPT Byggaren behöver för att fortsätta arbetet.

Det gör också att du kan byta konversation utan att återskapa historiken. Starta GPT Byggaren igen, bifoga den senaste projekt-ZIP:en och be den fortsätta projektet.

Det viktiga är alltså inte att bevara en lång chatt, utan att fortsätta från **rätt projektversion**.

När användartestning avslöjar ett större problem

Ibland visar testerna inte bara ett litet fel utan att själva idén behöver justeras.

Kanske försöker GPT:n lösa för många olika uppgifter.

Kanske behöver den mer kunskap än du först trodde.

Kanske kräver resultatet en mänsklig bedömning som inte bör automatiseras.

Eller kanske två olika användargrupper egentligen behöver olika assistenter.

Då är det bättre att ändra projektets inriktning än att försöka laga allt med fler instruktioner.

Exempel:

Vår GPT ska både göra en snabb första sortering av inkommande dokument och skriva en fullständig konsekvensanalys.

Efter testning kanske det visar sig att dessa två arbetsuppgifter kräver olika detaljnivå, olika frågor och olika resultatformat.

En möjlig förbättring är då att dela upp arbetsflödet eller skapa två tydliga lägen.

GPT Byggaren kan hjälpa till med en sådan omplanering, men beslutet om vad verksamheten faktiskt behöver är fortfarande ditt.

När är en version tillräckligt bra?

Det finns ingen generell punkt där en GPT blir ”färdig för alltid”.

En rimlig första version är ofta tillräckligt bra när:

- den löser sitt huvudsakliga användningsfall,
- de viktigaste gränssnitten har provats,
- kända begränsningar är begripliga,
- resultatet är konsekvent nog för målgruppen,
- allvarliga fel inte återkommer i regressionstester,
- användaren förstår vad assistenten kan och inte kan göra.

Det är ofta bättre att nå en avgränsad, fungerande version än att fortsätta lägga till funktioner bara för att de är möjliga.

En bra specialiserad assistent behöver inte kunna allt.

Den behöver vara **pålitligt användbar för sitt uppdrag**.

Ett praktiskt förbättringsvarv

När du har en egen GPT kan du använda följande enkla rutin:

1. Välj tre till fem realistiska uppgifter.
2. Kör dem som en vanlig användare.
3. Skriv ner de viktigaste problemen du ser.
4. Prioritera ett eller ett fåtal problem åt gången.
5. Beskriv situation, observation och önskat beteende för GPT Byggaren.
6. Låt GPT Byggaren uppdatera projektet.
7. Prova både det förbättrade fallet och några tidigare fall igen.
8. Spara den nya projekt-ZIP:en.

Detta behöver inte bli en stor testorganisation.

För en mindre GPT kan några väl valda verkliga exempel ge mycket värdefull information.

Det viktigaste från kapitlet

Att bygga en GPT är inte en engångsaktivitet där en perfekt instruktion skrivs från början.

Arbetet blir bättre om du tänker iterativt:

prova verkliga uppgifter, observera beteendet och förbättra det viktigaste först.

GPT Byggaren tar hand om mycket av projektförändringen, teststrukturen och paketeringen. Din viktigaste uppgift är att kunna se om assistenten verkligen hjälper användaren på rätt sätt.

När projektet ligger i sin projekt-ZIP kan utvecklingen dessutom fortsätta i en ny konversation utan att du behöver återskapa hela historiken.

I nästa kapitel lämnar vi utvecklingsloopen och tittar på nästa fråga: **när är en GPT ett experiment, när är den ett riktigt arbetsverktyg och vilken distributionsform passar då bäst?**

7. Från experiment till riktig assistent

Du har nu gått från en idé till en GPT som går att prova, förbättra och fortsätta utveckla.

Nästa fråga är inte främst teknisk:

Hur ska assistenten faktiskt användas och förvaltas?

En GPT kan vara allt från ett personligt experiment till ett arbetsverktyg som används av många personer. Ju viktigare assistenten blir, desto mer behöver du tänka på distribution, versionshantering, ansvar och hur förändringar görs.

Det betyder inte att varje GPT måste bli ett stort IT-projekt. Tvärtom är en av styrkorna med arbetssättet att du kan börja litet och höja ambitionsnivån först när behovet finns.

Börja med det enklaste som fungerar

Det är lätt att tänka att en ”riktig” GPT måste publiceras, integreras och automatiseras.

Så behöver det inte vara.

Om du har byggt en assistent för ett begränsat användningsfall och den fungerar bra i en vanlig ChatGPT-konversation kan **Chat ZIP** vara fullt tillräckligt.

Arbetsflödet kan då vara:

1. användaren startar en ny ChatGPT-konversation,
2. bifogar Chat ZIP-filen,
3. ber ChatGPT använda den som assistent,
4. genomför sitt arbete,
5. startar en ny konversation nästa gång det behövs.

Det är enkelt, portabelt och kräver ingen separat installation av en Custom GPT.

För en assistent som främst används av dig själv eller av ett mindre antal personer kan detta vara en mycket rimlig slutpunkt.

Välj inte en mer avancerad distributionsform bara för att den finns. Välj den när den löser ett verkligt problem.

När Chat ZIP passar bra

Chat ZIP är särskilt användbar när du vill kunna bära med dig assistentens instruktioner och stödmaterial som en fil.

Den passar ofta bra när:

- assistenten används av ett mindre antal personer,
- användaren kan starta arbetet genom att bifoga en ZIP-fil,
- projektet innehåller många filer eller rik runtime-struktur,
- du vill kunna prova nya versioner utan att ändra en publicerad GPT,
- du vill ha en portabel distribution som går att använda i nya konversationer.

Chat ZIP är också ett bra format under utveckling. Du kan bygga en version, använda den på riktiga uppgifter och sedan återvända till GPT Byggaren med dina observationer.

Det gör gränsen mellan experiment och användbart arbetsverktyg mindre dramatisk. Samma distributionsform kan fungera i båda lägena.

När Custom GPT kan vara bättre

En **Custom GPT** kan vara lämpligare när du vill att användaren ska kunna öppna en färdig assistent direkt i ChatGPT utan att först bifoga en Chat ZIP.

Det kan exempelvis vara värdefullt när:

- många personer ska använda samma assistent,
- du vill göra starten så enkel som möjligt,
- assistenten ska ha ett tydligt namn och en tydlig ingång,
- användarna inte behöver se eller hantera projektfiler,
- den funktionalitet som behövs ryms inom Custom GPT-plattformens möjligheter och begränsningar.

Det betyder inte att Custom GPT alltid är ”bättre” än Chat ZIP.

De är två olika sätt att distribuera samma grundidé.

För en liten GPT kan de vara nästan likvärdiga. För en mer omfattande assistent kan Chat ZIP ibland bära mer runtime-material än vad som är praktiskt eller möjligt att lägga i en Custom GPT.

GPT Byggaren är gjord för att hjälpa till med den bedömningen. Du behöver alltså inte bestämma distributionsform innan du ens vet hur GPT:n kommer att se ut.

Projekt-ZIP:en finns kvar bakom distributionen

Oavsett om användarna arbetar med Chat ZIP eller Custom GPT är **projekt-ZIP:en** fortfarande viktig.

Den är utvecklingsprojektet bakom den version som används.

En användare behöver kanske aldrig se den. Men den behövs när du vill:

- rätta ett problem,
- lägga till Knowledge,
- ändra ett arbetsflöde,
- förbättra tester,
- bygga en ny version,
- skapa en ny distribution.

Det är samma skillnad som mellan en färdig applikation och dess källkod.

Användaren arbetar med den distribuerade produkten. Den som förvaltar produkten behöver utvecklingsprojektet.

När versionshantering börjar bli värdefull

Så länge du experimenterar själv kan det räcka att spara den senaste projekt-ZIP:en med ett tydligt filnamn.

Men när assistenten börjar användas på riktigt uppstår snabbt frågor som:

- Vilken version använder vi?
- Vad ändrades sedan förra versionen?
- Kan vi gå tillbaka om en förändring blev dålig?
- Vilken ZIP hör till vilken release?
- Är den version vi testat samma som den vi har distribuerat?

Då blir versionshantering värdefull.

Nya GPT Byggaren-projekt har normalt GitHub-stöd från början, med bland annat README samt arbetsflöden för CI och release. Det går att välja bort för uttryckligen lokala eller GitHub-fria projekt. Med GitHub kan projektet exempelvis få:

- historik över förändringar,

- pull requests för granskning,
- automatiska tester vid förändringar,
- versionsmärkta releaser där versionen kan härledas från release-taggen,
- automatiskt byggda distributionsfiler.

Du behöver inte införa allt detta för din första GPT.

Men när fler människor blir beroende av assistenten är det bra att kunna svara på en enkel fråga:

Vilken version är det egentligen vi använder?

En enkel mognadstrappa

Du kan se utvecklingen som några naturliga nivåer.

1. Idé

Du har identifierat en återkommande uppgift som kanske lämpar sig för en specialiserad assistent.

2. Experiment

Du bygger en första version med GPT Byggaren och provar den själv.

Målet är lärande, inte perfektion.

3. Användbar assistent

GPT:n fungerar tillräckligt bra för verkliga uppgifter. Du känner till dess viktigaste begränsningar och har provat typiska användningsfall.

Chat ZIP kan mycket väl vara rätt distributionsform här.

4. Förvaltad arbetsverktyg

Fler personer använder assistenten eller den börjar få större betydelse. Versioner, tester, förändringshistorik och tydligt ansvar blir viktiga.

Här kan Custom GPT, Git och GitHub bli relevanta beroende på situationen.

5. Integrerad AI-lösning

Till sist kan behovet växa bortom vad en GPT är bäst lämpad för.

Du kanske behöver:

- integration med verksamhetssystem,
- automatisk behandling av stora mängder ärenden,
- strikt behörighetsstyrning,
- egna datalager,
- transaktioner och säkra uppdateringar i andra system,
- avancerad övervakning,
- garanterade svarstider eller andra driftkrav.

Då börjar frågan handla mindre om att konfigurera en GPT och mer om att bygga ett eget AI-baserat system.

Det är inte ett misslyckande för GPT:n. Tvärtom kan GPT-experimentet ha hjälpt dig förstå behovet innan du investerar i en större lösning.

När ska du inte bygga vidare på GPT:n?

Det är lika viktigt att kunna stanna.

En specialiserad GPT är särskilt bra när uppgiften handlar om språk, analys, sammanställning, vägledning och arbete med dokument eller information.

Men den är inte automatiskt rätt lösning på varje problem.

Var vaksam om användningsfallet börjar kräva att assistenten självständigt ska:

- fatta beslut med stora konsekvenser,
- garantera att varje svar är korrekt,
- ersätta obligatorisk mänsklig kontroll,
- hålla permanent verksamhetskritisk status utan lämpligt systemstöd,
- fungera som ett transaktionssystem,
- utföra uppgifter där fel inte kan tolereras eller fångas upp.

I sådana fall kan GPT:n fortfarande vara ett stöd, men den bör inte ensam bära hela processen.

Vem ansvarar för assistenten?

När en GPT går från personligt experiment till gemensamt arbetsverktyg behöver någon äga frågan:

Vem avgör vad assistenten ska göra och när den ska ändras?

Det behöver inte vara en formell förvaltningsorganisation för en liten GPT. Men det bör vara tydligt vem som:

- tar emot förbättringsförslag,
- bedömer förändringar i uppdraget,
- håller Knowledge aktuell,
- beslutar när en ny version ska tas i bruk,
- följer upp problem som användarna hittar.

GPT Byggaren kan hjälpa till att genomföra förändringarna. Den kan inte ersätta verksamhetens ansvar för vad assistenten ska användas till.

Ett praktiskt vägval

Anta att remissassistenten från tidigare kapitel nu fungerar bra.

Du använder den själv några gånger i månaden och vill framför allt kunna utveckla den vidare när du upptäcker förbättringar.

Då kan en rimlig lösning vara:

Behåll projekt-ZIP:en för utveckling och använd Chat ZIP i det dagliga arbetet.

Några månader senare börjar tio kollegor använda samma assistent. De vill slippa bifoga en ZIP varje gång och ni behöver kunna säga vilken version som är den aktuella.

Då kan nästa steg vara:

Bygg en Custom GPT för användarna och lägg utvecklingsprojektet i GitHub med versionsmärkta releaser.

Senare kanske organisationen vill att remisser automatiskt ska hämtas från ett ärendehanteringssystem, analyseras och kopplas tillbaka till rätt ärende.

Då har behovet förändrats igen.

Nu kan en integrerad AI-lösning vara mer lämplig än att fortsätta pressa in allt i själva GPT:n.

Poängen är att tekniken får följa behovet, inte tvärtom.

Din assistent behöver inte nå sista nivån

Mognadstrappan är inte en tävling.

En personlig Chat ZIP som löser ett verkligt problem kan vara en helt färdig lösning.

En Custom GPT som används av ett arbetslag behöver inte bli ett separat IT-system.

Och ett experiment som visar att idén inte ger tillräcklig nytta kan också vara ett bra resultat.

Det viktiga är att välja **minsta ambitionsnivå som ger den nytta du faktiskt behöver**.

Det viktigaste från kapitlet

När en GPT fungerar behöver du inte automatiskt göra den mer avancerad.

Tänk i stället i tre frågor:

1. **Hur ska den användas?** – räcker Chat ZIP eller blir Custom GPT enklare för användarna?
2. **Hur viktig har den blivit?** – behöver projektet versionshantering, tydligare tester och en förvaltare?
3. **Har behovet vuxit bortom en GPT?** – är det dags för en integrerad AI-lösning i stället?

GPT Byggaren gör det möjligt att börja enkelt och utveckla lösningen stegvis.

Det sammanfattar också bokens huvudidé: börja med **problemet, användaren och ett bra resultat**, och låt tekniken växa först när behovet kräver det.

idé → analys → plan → projekt → prova → förbättra → distribuera

Det är skillnaden mellan en bra prompt för stunden och en AI-assistent som går att använda och vidareutveckla.